

# 计算机系统基础实 验报告

实验三：Coroutinelab



学生姓名：俞浩坤

学    号：22300240034

日    期：2023.11.3

## 一、part0:

任务1:将函数代码复制到 test.cpp 后,输入 `chmod +x test`,然后输入 `gdb ./test` 可以用 gdb 打开 test 文件。

```
yuhaoshen@LAPTOP-QU516I60:/mnt/c/bomblab-handout$ chmod +x test
yuhaoshen@LAPTOP-QU516I60:/mnt/c/bomblab-handout$ ./test
42
yuhaoshen@LAPTOP-QU516I60:/mnt/c/bomblab-handout$ gdb ./test
GNU gdb (Ubuntu 12.1-0ubuntu1~22.04) 12.1
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./test...
```

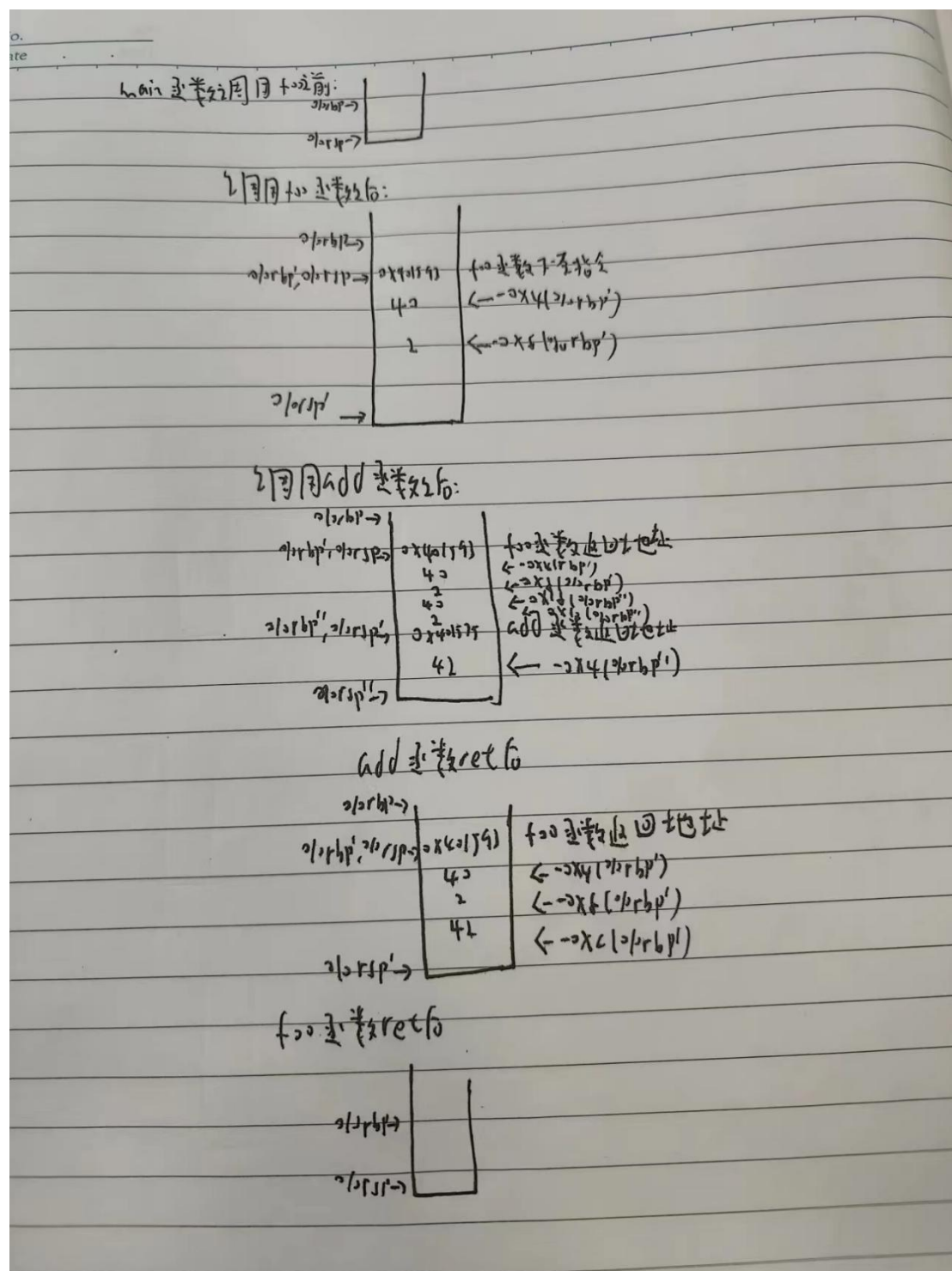
采用 `disas` 指令可以查看三个函数的汇编代码

```
Dump of assembler code for function main:
0x0000000000401581 <+0>:    push    %rbp
0x0000000000401582 <+1>:    mov     %rsp,%rbp
0x0000000000401585 <+4>:    sub     $0x30,%rsp
0x0000000000401589 <+8>:    call    0x40e7a8 <__main>
0x000000000040158e <+13>:   call    0x401552 <_Z3foov>
0x0000000000401593 <+18>:   mov     %eax,-0x4(%rbp)
0x0000000000401596 <+21>:   mov     -0x4(%rbp),%eax
0x0000000000401599 <+24>:   mov     %eax,%edx
0x000000000040159b <+26>:   mov     0x8b0ee(%rip),%rcx    # 0x48c690 <.refptr._ZSt4cout>
0x00000000004015a2 <+33>:   call    0x44d558 <_ZNSolsfi>
0x00000000004015a7 <+38>:   mov     0x8b0f2(%rip),%rdx    # 0x48c6a0 <.refptr._ZSt4endlcIcSt11char_traitsIcEERSt13
basic_ostreamIT_7_E6>
0x00000000004015ae <+45>:   mov     %rax,%rcx
0x00000000004015b1 <+48>:   call    0x44d358 <_ZNSolsEPFRSoS_E>
0x00000000004015b6 <+53>:   mov     $0x0,%eax
0x00000000004015bb <+58>:   add     $0x30,%rsp
0x00000000004015bf <+62>:   pop     %rbp
0x00000000004015c0 <+63>:   ret
```

```
Dump of assembler code for function _Z3foov:
0x0000000000401552 <+0>:    push    %rbp
0x0000000000401553 <+1>:    mov     %rsp,%rbp
0x0000000000401556 <+4>:    sub     $0x30,%rsp
0x000000000040155a <+8>:    movl    $0x28,-0x4(%rbp)
0x0000000000401561 <+15>:   movl    $0x2,-0x8(%rbp)
0x0000000000401568 <+22>:   mov     -0x8(%rbp),%edx
0x000000000040156b <+25>:   mov     -0x4(%rbp),%eax
0x000000000040156e <+28>:   mov     %eax,%ecx
0x0000000000401570 <+30>:   call    0x401530 <_Z3addii>
0x0000000000401575 <+35>:   mov     %eax,-0xc(%rbp)
0x0000000000401578 <+38>:   mov     -0xc(%rbp),%eax
0x000000000040157b <+41>:   add     $0x30,%rsp
0x000000000040157f <+45>:   pop     %rbp
0x0000000000401580 <+46>:   ret
```

```
(gdb) disas add
Dump of assembler code for function _Z3addii:
0x0000000000401530 <+0>:    push    %rbp
0x0000000000401531 <+1>:    mov     %rsp,%rbp
0x0000000000401534 <+4>:    sub     $0x10,%rsp
0x0000000000401538 <+8>:    mov     %ecx,0x10(%rbp)
0x000000000040153b <+11>:   mov     %edx,0x18(%rbp)
0x000000000040153e <+14>:   mov     0x10(%rbp),%edx
0x0000000000401541 <+17>:   mov     0x18(%rbp),%eax
0x0000000000401544 <+20>:   add     %edx,%eax
0x0000000000401546 <+22>:   mov     %eax,-0x4(%rbp)
0x0000000000401549 <+25>:   mov     -0x4(%rbp),%eax
0x000000000040154c <+28>:   add     $0x10,%rsp
0x0000000000401550 <+32>:   pop     %rbp
0x0000000000401551 <+33>:   ret
```

函数调用过程分析：从 main() 中调用 foo 函数开始，函数首先将调用前栈顶指针存入 %rbp 中，然后将栈指针减少 0x30 分配空间，接着将 foo 函数中的局部变量 40 和 2 存入新开的空间中，即 -0x4(%rbp) 和 -0x8(%rbp)，再将它们存入寄存器中用寄存器传递参数，之后再调用 add 函数，先将调用前的 %rbp 值压入栈中保存，并减少栈顶指针分配空间，再从寄存器中将参数取出然后相加，将结果存储在 %eax 中，之后增加栈顶指针释放空间，最后把 %rbp 弹出使其恢复原位置，调用完 add 函数后通过 -0xc(%rbp) 将值再传回 %eax，之后释放空间弹出 %rbp 使其恢复原位，函数调用结束。



## 任务 2:

1、一个“普通”的函数支持哪两个操作，分别承担了什么功能？

一个“普通”的函数支持 Call 和 Return 两个操作:

Call 操作创建一个激活帧，暂停调用的执行函数并将执行转移到被调用函数的开头。

Return 操作将返回值传递给调用方，销毁激活帧 然后，在调用方调用函数的点之后继续执行调用方。

2、为什么我们说调用栈不能满足协程的要求？

1)、由于协程可以在不破坏激活帧的情况下挂起，因此我们不能再保证激活帧的生存期是严格嵌套的。这意味着激活帧通常不能使用堆栈数据结构进行分配，因此可能需要将其存储在堆中。

2)、协程能够做到在函数的任意位置暂停或者恢复，然而调用栈有明确的出口和入口，因而无法满足协程的要求。

3)、函数在返回后调用栈中所有存储的变量都将被销毁，但协程在返回后仍然会保存其参数状态，只会销毁局部变量。

3、协程作为一种泛化的函数，支持了哪几个操作，分别承担了什么功能？

协程通过将 Call 和 Return 操作中执行的一些步骤分离，支持三个额外的操作:Suspend, Resume 和 Destroy。

1)、Call:协程的 Call 操作与普通函数的 Call 操作非常相似。然而，与只在函数运行到完成时才返回调用方的执行不同，对于协程，Call 操作将在协程到达第一个挂起点时恢复调用方的执行。在协程上执行 Call 操作时，调用者分配一个新的堆栈帧，将参数写入堆栈帧，将返回地址写入堆栈帧，并将执行转移到协程。协程做的第一件事是之后在堆上分配协程帧，并将参数从堆栈帧复制/移动到协程帧中，以便参数的生存期超出第一个挂起点。

2)、Suspend: 协程的 Suspend 操作允许协程在函数的中间暂停执行，并将执行转回给协程的调用者或恢复者。协程主体中有一些点被指定为挂起点。当协程遇到这些挂起点之一时，它首先准备协程，以便通过以下方式恢复:

确保寄存器中保存的任何值都被写入协程帧;

向协程帧写入一个值，该值指示协程被挂起在哪个挂起点。这允许后续的 Resume 操作知道在哪里恢复协程的执行，或者后续的 Destroy 操作知道哪些值在作用域中并且需要销毁。

一旦协程准备好恢复，则认为该协程“Suspend”。

3)、Resume: Resume 操作可以在当前处于“Suspend”状态的协程上执行。当一个函数想要恢复一个协程时，它需要有效地“调用”到函数的特定调用的中间。

resume 识别要恢复的特定调用的方法是调用提供给相应 Suspend 操作的协程框架句柄上的方法。就像普通的函数调用一样，这个调用将分配一个新的堆栈帧，并在将执行转移到函数之前在堆栈帧中存储调用者的返回地址。然而，它不是将执行转移到函数的开始，而是将执行转移到函数中最后挂起的点。它通过从协程框架加载恢复点并跳转到该点来实现这一点。当协程下一次挂起或运行到完成时，这个调用将返回并继续执行调用函数。

4)、Destroy: Destroy 操作销毁协程帧，但不恢复协程的执行。此操作只能在挂起的协程上执行。Destroy 操作的行为很像 Resume 操作，因为它重新激活协程的激活帧，包括分配一个新的堆栈帧和存储 Destroy 操作调用者的返回地址。但是，它不是在最后一个挂起点将执行转移到协程体，而是将执行转移到另一个



代码路径,该代码路径在释放协程框架使用的内存之前,调用挂起点范围内所有局部变量的析构函数。与 Resume 操作类似,Destroy 操作通过调用相应的 Suspend 操作期间提供的协程框架句柄上的方法来确定要销毁的特定激活帧。

5)、Return: 协程的 Return 操作与普通函数的 Return 操作略有不同。当协程执行操作时,它将返回值存储在某个地方(具体存储位置可以由协程自定义),然后销毁任何作用域内的局部变量(但不包括参数)协程然后有机会执行一些额外的逻辑,然后再将执行转回调用者/接收者。这个额外的逻辑可能会执行一些操作来发布返回值,或者它可能会恢复正在等待结果的另一个协程。它是完全可定制的。协程然后执行 Suspend 操作(保持协程帧活动)或 Destroy 操作(销毁协程帧)。

然后根据 Suspend/Destroy 操作语义将执行转回调用者/恢复程序,将激活帧的堆栈帧组件从堆栈中弹出。

4、如果不能用栈来实现协程,那么我们可以将函数运行时所需的信息存储在哪里?

如果编译器能够证明协程的生命周期确实严格嵌套在调用者的生命周期内,则允许从调用者的激活帧中分配协程帧的内存,否则数据会被存储在堆中。

任务 3: CPU 的状态包括寄存器、程序计数器以及堆栈指针等,为了暂停并恢复一个函数的运行,我们需要保存如下状态:

1、寄存器: 函数的暂停需要保存当前函数运行过程中使用到的寄存器的值,以便在恢复时能够保持函数内部变量的状态不变。

2、程序计数器: 函数的暂停需要记录下一条将要执行指令的地址,以便恢复时能够从正确的位置继续执行。

3、堆栈指针以及暂停时栈中存有的数据: 函数的暂停需要保存当前栈指针的位置以及栈中所存有的数据,以便在恢复时能够回到暂停前的栈状态。

## 二、part1:

coro.h 部分:

coroutine 部分:

```
coroutine* create(func_t func, void* args);
void release(coroutine* co);
int resume(coroutine* co, int param = 0);
int yield(int ret = 0);
const int maxsize = 999999;

struct coroutine {
    bool started = false;
    bool end = false;

    func_t coro_func = nullptr;
    void* args = nullptr;

    int midnum = 0;

    ucontext_t ctx = { 0 };
    int corophase;
    char corostack[maxsize];
    coroutine(func_t func, void* args) : coro_func(func), args(args) {}
    ~coroutine() {}
};
```

在模板的基础上新添加了一个数组来存储上下文结构体信息，同时增加 corophase 变量来记录协程的状态，在之后将用 1 来表示协程准备完成，用 2 表示协程运行中，用 3 表示协程结束，并根据提示在结构体中设置 midnum 字段用于 resume 和 yield 之间的通信。

coroutine\_env 部分：

```
class coroutine_env {
private:
    int depth = 0;
    coroutine* coroall[2];
public:
    int midnum = 0;
    ucontext_t main_coro;
    coroutine_env() {}

    coroutine* get_coro(int idx)
    {
        assert(idx == 0 || idx == 1);
        return coroall[idx];
    }

    void push(coroutine* co)
    {
        assert(depth < 2);
        coroall[depth] = co;
        depth++;
    }

    coroutine* pop()
    {
        assert(depth >= 1);
        coroutine* co = coroall[depth - 1];
        depth--;
        return co;
    }
};
```

在 coroutine\_env 结构体中，压入主协程 main\_coro 来记录 main 函数的上下文，由于栈的深度可以只设置为 2，因而只需开两个空间来记录协程调用信息，get\_coro 函数只需返回其参数在数组中对应的位置，push 函数则将参数协程压入数组尾部，pop 函数把数组尾元素存储入 co 中后使深度减 1，以此使下一次压入时能在原位置赋值，在进行函数操作前需要判断深度和参数的大小以免发生内存错误。

coro.cpp 部分：

create 函数部分：

```
coroutine* create(func_t func, void* args)
{
    coroutine* latecoro = new coroutine(func, args);
    latecoro->corophase = 1;
    latecoro->ctx.uc_stack.ss_sp = latecoro->corostack;
    latecoro->ctx.uc_stack.ss_size = maxsize;
    return latecoro;
}
```

create 函数采用构造函数给协程分配空间，然后将其状态设置为 1 表示准备完成，将其上下文结构体存储的位置定为之前新开的数组，并指定其大小。

release 函数部分：

```

void release(coroutine* co)
{
    if (co != 0)
    {
        delete co;
    }
    return;
}

```

release 函数直接判断协程是否存在，如果存在则删除该协程释放空间。  
resume 函数部分：

```

int resume(coroutine* co, int param)
{
    g_coro_env.push(co);
    co->midnum = param;
    switch (co->corophase)
    {
        case 1:
            getcontext(&co->ctx);
            co->ctx.uc_link = &g_coro_env.main_coro;
            co->corophase = 2;
            makecontext(&co->ctx, (void (*)(void)) func_wrap, 1, co);
            swapcontext(&g_coro_env.main_coro, &co->ctx);
            break;
        case 2:
            swapcontext(&g_coro_env.main_coro, &co->ctx);
            break;
    }
    return co->midnum;
}

```

resume 函数因为运行的协程总是位于栈顶，因而要恢复协程的运行首先要把当前协程压入栈，如果恢复的协程已经在运行了，则采用 swapcontext 函数将主协程上下文保存然后直接设置当前上下文为要恢复协程的上下文，如果还没运行的话则首先用 getcontext 函数获取上下文并储存，再将该上下文与主协程链接，表示当前协程运行完后会切换到主协程的上下文，将其状态修改为运行中后再采用 makecontext 函数，在 makecontext 函数中调用辅助 func\_wrap 函数同一协程的进入，并在协程退出时设置相关标志位，最后同样采用 swapcontext 函数进行切换。函数中还用结构体中的 midnum 字段保留了传递给协程的参数，以便 yield 函数能够返回。

func\_wrap 函数部分：

```

static void func_wrap(coroutine* co)
{
    if (co->coro_func)
    {
        co->coro_func(co->args);
    }
    co->end = true;
    co->corophase = 3;
    yield(-1);
}

```

func\_wrap 函数中由于不知道如何把协程准备完成和协程运行中区分开来，因而没有使用模板给出的 started 和 end 成员，而是自己新开了 corophase 成员，因而在函数中也要做出相应修改，使函数执行结束后状态设置为结束。

yield 函数部分:

```
int yield(int ret)
{
    coroutine* co = g_coro_env.pop();
    co->midnum = ret;
    if (ret != -1)
    {
        swapcontext(&co->ctx, &g_coro_env.main_coro);
        return co->midnum;
    }
    return -1;
}
```

yield 函数由于要暂停协程的进行因而直接将栈顶协程弹出并存储在 co 中，然后判断输入是否为-1，如果是则协程结束，不是的话采用 swapcontext 函数进行切换。同样的，函数中采用结构体中 midnum 字段保留参数以便 resume 函数能够返回。

代码实现完成后，进入 libco/libco\_v1 文件夹下用 make 命令编译代码，使用 ./main 命令运行：

```
yuhaoshen@LAPTOP-QU516I60:/mnt/c/libco-handout/libco - TODO/libco_v1$ make
g++-11 -std=c++20 -o main main.cpp coro.cpp
yuhaoshen@LAPTOP-QU516I60:/mnt/c/libco-handout/libco - TODO/libco_v1$ ./main
test-1 passed
test-2 passed
test-3 passed
Congratulations! You have passed all the tests of libco-v1!
```

运行结果一切正常，完成 part1。

### 三、part2 (coro\_ctx) :

coro\_ctx 结构体:

coro\_ctx\_swap 函数部分:

```
leaq (%rsp),%rax    //rsp -> rax
movq %rax, 104(%rdi) //rsp
movq %rbx, 96(%rdi)  //rbx
movq %rcx, 88(%rdi)  //rcx
movq %rdx, 80(%rdi)  //rdx
movq 0(%rax), %rax
movq %rax, 72(%rdi)  //ret addr
movq %rsi, 64(%rdi)  //rsi
movq %rdi, 56(%rdi)  //rdi
movq %rbp, 48(%rdi)  //rbp
movq %r8, 40(%rdi)   //r8
movq %r9, 32(%rdi)   //r9
movq %r12, 24(%rdi)  //r12
movq %r13, 16(%rdi)  //r13
movq %r14, 8(%rdi)   //r14
movq %r15, (%rdi)    //r15
xorq %rax, %rax//    return 0
```



```

movq 48(%rsi), %rbp    //rbp
movq 104(%rsi), %rsp   //rsp
movq (%rsi), %r15      //r15
movq 8(%rsi), %r14     //r14
movq 16(%rsi), %r13    //r13
movq 24(%rsi), %r12    //r12
movq 32(%rsi), %r9     //r9
movq 40(%rsi), %r8     //r8
movq 56(%rsi), %rdi    //rdi
movq 80(%rsi), %rdx    //rdx
movq 88(%rsi), %rcx    //rcx
movq 96(%rsi), %rbx    //rbx
leaq 8(%rsp), %rsp     //rsp += 8
pushq 72(%rsi)        // push ret addr

movq 64(%rsi), %rsi    // rsi
ret

```

参考了助教在实验课上给出的在 windows 环境下的汇编代码，在切换上下文时，保存传递参数的 6 个寄存器，栈指针和函数地址，再将当前寄存器的值，栈指针和函数地址切换为下一个上下文对应的值，由于在 linux 环境下函数对应的第一个和第二个参数是 rdi 和 rsi 因而要将在 windows 操作系统下相对于 rcx 和 rdx 的偏移量改为相对于 rdi 和 rsi 的偏移量，代码中每次保存和切换的内容已在注释中给出。

coro\_ctx.h 部分：

```

struct context {
    void *regs[14];
    int ss_size;
    void* ss_sp;
};

```

context 结构体需要一个大小为 14 的寄存器数组保存汇编代码中需要的值，还需要 ss\_size 成员和 ss\_sp 指针分别表示栈上剩余空间的大小和栈指针 rsp 的位置以此来保存上下文。

coro\_ctx.cpp 部分：

```

void ctx_make(context* ctx, func_t coro_func, const void* arg) {
    for(int i = 0 ; i < 14 ; i++)
    {
        ctx->regs[i] = 0;
    }
    char* sp = (char*)ctx->ss_sp + ctx->ss_size - sizeof(void*);
    sp = (char*)((uintptr_t)sp & -16LL);
    ctx->regs[7] = (void*) arg;
    ctx->regs[9] = (void*) coro_func;
    ctx->regs[13] = sp;
}

```

ctx\_make 函数首先对上下文的寄存器数组初始化为 0，接着将新的栈指针指向原来的栈指针加上栈内剩余空间的位置，之后根据助教在群里给出的提示，需要对

指针进行 16 位字节对齐，最后处理需要保存的内容在寄存器数组中的位置，其中 regs[0]-regs[7] 保存 r15, r14, r13, r12, r9, r8, rbp 和 rdi8 个寄存器，以函数第一个参数 rdi 为截止，regs[8] 和 regs[9] 保存 rsi 和函数返回地址 ret，由于一个上下文只要一个函数参数因而第二个参数 rsi 实际不会用到，最后 regs[10]-regs[13] 保存 rdx, rcx, rbx 和 rsp8 个寄存器，以栈指针 rsp 为截止。

## 四、part2 (coroutine) 和 part2.5:

由于 part2.5 中会修改 part2 中 coroutine 部分的代码，因而选择将两者的报告结合在一起。

coroutine.h:

stack\_mem 部分:

```
struct stack_mem {
    int stack_size = 0;           // 栈的大小
    coroutine *owner = nullptr;
    char *buf = nullptr;
    char *bp = nullptr;

    stack_mem(size_t size) : stack_size(size) {
        buf = new char[size];
        bp = buf + size;
    }

    ~stack_mem() {
        delete[] buf;
    }
};
```

stack\_mem 结构中除了需要栈的大小外，由于多个协程共享一个栈结构，因而需要一个 owner 成员保存当前使用栈的协程，同时在协程切换时需要保存当前共享栈的内容，因而需要一个 buf 成员来储存，还需要一个 bp 成员作为共享栈的栈指针。构造函数中给 buf 一段参数大小的空间，并更新栈指针的位置，析构函数将该段空间用 delete 释放即可。

share\_stack 部分:

```
struct share_stack {
    int nextidx = 0;
    int count = 0;
    int stack_size = 0;
    stack_mem **stack_array = nullptr;

    share_stack(int count, size_t stack_size)
        : count(count), stack_size(stack_size) {
        stack_array = new stack_mem*[count];
        for (int i = 0; i < count; i++)
        {
            stack_array[i] = new stack_mem(stack_size);
        }
    }

    ~share_stack() {
        for (int i = 0; i < count; i++)
        {
            delete stack_array[i];
        }
        delete[] stack_array;
    }

    stack_mem *get_stackmem() {
        int idx = nextidx % count;
        nextidx++;
        return stack_array[idx];
    }
};
```

由于 share\_stack 中的 get\_stackmem 函数需要取出当前共享栈，因而需要多加入一个 nextidx 成员来定位其在数组中的位置，构造函数中调用 stack\_mem 构造

函数,给数组每个成员均分配一个参数大小的空间,数组大小由第一个参数给出,析构函数则用 delete 释放数组每一个成员所占用的空间,在 get\_stackmem 函数中由于要减少拷贝带来的开销,因而实际协程数可能比 count 要大,在这种情况下为了充分利用每个共享栈,我们将下标对 count 取模,使存储协程信息数量最大的共享栈最小,同时让记录索引的 nextidx 成员自增。

coroutine 部分:

```
struct coroutine {
    bool started = false;
    bool end = false;

    func_t coro_func = nullptr;
    void *arg = nullptr;

    int midnum = 0;

    context ctx = {0};

    stack_mem* coro_stack_mem = nullptr;
    char* sp = nullptr;
    int usedsize = 0;
    char* usedbuf = nullptr;
    int stack_size = 0;

    coroutine(func_t func, void* arg) : coro_func(func), arg(arg) {}
    ~coroutine() {}
};
```

在 part1 的基础上新增加了一个 coro\_stack\_mem 栈成员来存储当前协程的信息,并加入 sp 成员记录栈顶指针,usedsize 成员记录使用到的栈的大小,以及 usedbuf 成员储存使用到的栈的内容,以便于在拷贝时根据协程使用到的空间拷贝,以此来节省开销,同时还需要一个 stack\_size 成员来记录栈的总大小,注意到由于在 coroutine.cpp 中实现 part1 中的四个同名函数时不能使用 ucontext 的库函数,而是要自己实现,在没有库函数的使用限制后发现只需要模板中的 started 和 end 成员就可以实现函数功能,因而不需要 part1 中的 corophase 变量了。

coroutine\_env 部分:

```
class coroutine_env {
private:
    coroutine* coroall[128];
public:
    coroutine* latecoro = nullptr;
    coroutine* nowcoro = nullptr;
    int midnum = 0;
    int depth = 0;
    coroutine_env()
    {
        coroutine* main_coro = create(nullptr, nullptr, nullptr);
        push(main_coro);
    }

    coroutine* get_coro(int idx)
    {
        return coroall[idx];
    }

    void pop()
    {
        depth--;
    }

    void push(coroutine* co)
    {
        coroall[depth] = co;
        depth++;
    }
};
```

coroutine\_env 结构中根据题目的要求将 part1 中栈的深度扩大到 128，之后由于栈的深度变大了，因而需要增加 latecoro 和 nowcoro 两个新成员来记录下一个要切换到的协程和之前存储的协程，同时注意到栈的实际深度成员 depth 在之后 coroutine.cpp 的函数中也需要用到，因而要将它从 private 成员中换到 public 成员中去，同样由于栈深度的扩大，不能够直接单独的开一个主协程成员，而应在结构体初始化时直接创建一个空协程压入栈中，将主协程作为栈首元素以此来方便后续的操作，其余函数部分和 part1 中的实现大致相同，不同的是将 pop 函数改为了 void 类型而不需要返回值了。

coroutine.cpp:

create 函数部分:

```
coroutine* create(func_t coro_func, void* arg, const coroutine_attr* attr) {
    coroutine_attr at;
    if (attr != nullptr) {
        at = *attr;
    }

    if (at.stack_size < 8 * 1024)
    {
        at.stack_size = 8 * 1024;
    }
    if (at.stack_size > 8 * 1024 * 1024)
    {
        at.stack_size = 8 * 1024 * 1024;
    }
    if (at.stack_size & 0xFFF == 1)
    {
        at.stack_size = at.stack_size & ~0xFFF;
        at.stack_size = at.stack_size + 0x1000;
    }
    coroutine* latecoro = new coroutine(coro_func, arg);
    stack_mem* lateststack_mem = nullptr;
    if (at.sstack == nullptr)
    {
        lateststack_mem = new coro::stack_mem(at.stack_size);
        lateststack_mem->owner = latecoro;
    }
    else
    {
        lateststack_mem = at.sstack->get_stackmem();
    }
    latecoro->coro_stack_mem = lateststack_mem;
    latecoro->stack_size = latecoro->coro_stack_mem->stack_size;
    latecoro->ctx.ss_size = latecoro->stack_size;
    latecoro->ctx.ss_sp = latecoro->coro_stack_mem->buf;
    return latecoro;
}
```

create 函数首先根据提示使传入栈的大小能在 8k 到 128k 之间。

为了实现上面的操作，你需要修改 create 函数。你可能也需要在其中检测 attr 传入的数据是否合理，例如栈大小应该在 8k 到 128k 之间，否则强制修改到边界值。另外，最好让 stack\_size 为 4k 的倍数，可以通过位操作来检验和修改。

接着通过位操作使其变为 4k 的倍数，即判断它二进制后 12 位是否全为 0，如果不是则代表不是 4k 的倍数，则将其后 12 位置 0 后再加上 4k 使其向上取到 4k 的倍数，接着采用构造函数给协程分配空间，并给协程开一个栈空间，如果传入协程没有共享栈，则将调用构造函数对栈空间初始化，并将其大小设为传入的协程的栈大小，将其使用的协程定为新协程，否则直接调用 get\_stackmem 函数对其进行赋值，之后将协程的栈空间赋值为新开的栈空间，栈大小赋值为栈空间的栈大小，上下文的大小同样赋值为栈的大小，而上下文的栈指针则要赋值为栈使用空间的首地址表明信息存储的位置。

release 函数部分和 part1 一样不再赘述。

save\_stack 函数部分:



```

void save_stack(coroutine* co) {
    stack_mem* latest_stack_mem = co->coro_stack_mem;
    int len = latest_stack_mem->bp - co->sp;
    if (co->usedbuf)
    {
        delete[] co->usedbuf;
        co->usedbuf = nullptr;
    }
    co->usedbuf = new char[len];
    co->usedsize = len;
    memcpy(co->usedbuf, co->sp, len);
}

```

save\_stack 函数首先通过协程栈指针 bp 减去栈顶指针 sp 来得到协程栈的大小，接着如果协程存储栈内容的成员已经有内容了，则将其释放后再根据大小分配新的空间，然后用 memcpy 函数从栈顶指针开始存储栈的信息，同时也要更新栈的使用大小。

swap 函数部分：

```

void swap(coroutine* curr, coroutine* pending) {
    char c;
    curr->sp = &c;
    g_coro_env.latecoro = pending;
    g_coro_env.nowcoro = pending->coro_stack_mem->owner;
    pending->coro_stack_mem->owner = pending;
    if (g_coro_env.nowcoro && g_coro_env.nowcoro != pending)
    {
        save_stack(g_coro_env.nowcoro);
    }
    coro_ctx_swap(&(curr->ctx), &(pending->ctx));
    if (g_coro_env.nowcoro && g_coro_env.latecoro && g_coro_env.latecoro != g_coro_env.nowcoro)
    {
        if (g_coro_env.latecoro->usedbuf && g_coro_env.latecoro->usedsize > 0)
        {
            memcpy(g_coro_env.latecoro->sp, g_coro_env.latecoro->usedbuf, g_coro_env.latecoro->usedsize);
        }
    }
}

```

swap 函数首先给现在运行的协程的栈顶指针开一个空间，之后将需要切换到的协程 pending 用 g\_coro\_env 的 latecoro 成员保存，将 pending 使用栈空间中上一个使用的协程用另一个 nowcoro 成员保存，之后将栈空间当前使用的协程赋值为 pending，之后判断上一个使用的协程是否存在以及是否与现在要切换到的协程相同，如果存在且不相同则调用 save\_stack 函数存储其信息，之后调用 coro\_ctx\_swap 函数保存并切换上下文。最后如果上一个使用的协程以及要切换到的协程均存在且不相等，以及要切换到的协程存储栈内容的成员里有内容，则将内容用 memcpy 函数以存储到以栈顶指针为首地址的空间中。

func\_wrap 函数部分：

```

static void func_wrap(coroutine* co) {
    if (co->coro_func) {
        co->coro_func(co->arg);
    }
    co->end = true;
    yield(-1);
}

```

由于这次没有新开成员变量来记录协程状态，因而不需修改。

resume 函数部分：

```

int resume(coroutine* co, int param) {
    co->midnum = param;
    if (!co->started)
    {
        ctx_make(&co->ctx, (func_t)func_wrap, co);
        co->started = true;
    }
    g_coro_env.push(co);
    swap(g_coro_env.get_coro(g_coro_env.depth - 2), co);
    return co->midnum;
}

```

resume 函数中关于传递参数的部分和 part1 一样，接着判断当前恢复的协程是否开始运行了，如果没有则获取当前上下文并存储，接着标记为开始运行，之后将其压入栈顶，由于压入前运行的协程位于栈顶，因而压入后之前运行的协程现在是栈的第二个元素，用 swap 函数切换二者的上下文即可恢复协程的运行。

yield 函数部分：

```

int yield(int ret) {
    coroutine* curr = g_coro_env.get_coro(g_coro_env.depth - 1);
    g_coro_env.pop();
    curr->midnum = ret;
    swap(curr, g_coro_env.get_coro(g_coro_env.depth - 1));
    return curr->midnum;
}

```

因为要挂起的协程位于栈顶，因而可直接将其取出并用 curr 记录，之后用 swap 函数切换它与现在位于栈顶的协程的上下文即可将协程挂起。

代码实现完成后，进入 libco/libco\_v2 文件夹下用 make 命令编译代码，使用 ./main 命令运行：

```

yuhaoshen@LAPTOP-QU516I60:/mnt/c/libco-handout/libco - TODO/libco_v2$ make
g++-11 -g -std=c++20 -o main coro_ctx_swap.S coro_ctx.cpp coroutine.cpp main.cpp
yuhaoshen@LAPTOP-QU516I60:/mnt/c/libco-handout/libco - TODO/libco_v2$ ./main
test-1 passed
test-2 passed
test-3 passed
test-4 passed
Congratulations! You have passed all the tests of libco-v2!

```

运行结果正常通过测试，完成 part2 和 part2.5，至此完成实验中有栈协程的实现。

## 五、part3:

main.cpp 部分：

```

#define CO_BEGIN switch(_line) { case 0:
#define CO_END _line = __LINE__; case __LINE__: } return 0;
#define CO_YIELD(...) _line = __LINE__; return __VA_ARGS__; case __LINE__:;
#define CO_RETURN(...) { _done = true; return __VA_ARGS__; }

struct coroutine_base {
    int _line;
    bool _done;

    coroutine_base() : _line(0), _done(false) {}

    bool done() const { return _done; }
};

```

在 `coroutine_base` 类中加入 `_line` 成员来记录每次挂起的位置, 加入 `_done` 成员来记录协程进行是否结束, 在构造函数中初始化两者为 0 和 `false`, 在 `CO_BEGIN` 宏中通过执行 `_line` 的初值所对应代码来开始协程, 在 `CO_END` 宏中将 `_line` 重新赋值, 并把其对应的代码设置为空, 最后 `return 0`, 以此来结束协程, 在 `CO_YIELD` 宏中同样将 `_line` 重新赋值, 通过 `return _VA_ARGS_` 来返回函数参数, 再把其对应代码设置为空以便在下次运行时能从挂起位置继续运行, 在 `CO_RETURN` 中将协程结束标志设为 `true` 表明协程结束并返回函数参数。

```
class fib : public coroutine_base {
private:
    int a = 0;
    int b = 1;

public:
    int operator()()
    {
        CO_BEGIN;
        while (!_done)
        {
            CO_YIELD(a);
            int tmp = a;
            a = b;
            b = tmp + b;
        }
        CO_RETURN(-1);
        CO_END;
    }
};
```

在 `fib` 类中重载 `()` 操作符, 一开始先开始协程, 由于每次所需要的答案都是进入循环后一开始 `a` 的值, 因而在每次循环一开始暂停协程并返回 `a` 的值, 在循环外仿照 `yield(-1)` 以 `CO_RETURN(-1)` 来表示结束, 并在最后彻底结束协程, 但由于 `main` 函数中一共只会循环 10 次因而程序不会进行到循环外。

代码实现完成后, 进入 `libco/libco_v3` 文件夹下用 `make` 命令编译代码, 使用 `./main` 命令运行:

```
yuhaoshen@LAPTOP-QU516I60:/mnt/c/libco-handout/libco - TODO/libco_v3$ make
g++-11 -o main main.cpp
yuhaoshen@LAPTOP-QU516I60:/mnt/c/libco-handout/libco - TODO/libco_v3$ ./main
libco_v3 test passed!
```

通过 `main` 函数中的测试, 完成 part3。

## 六、part4:

任务 4:

1、协程函数的返回值 `Coroutine Functor` 需要有哪些成员?

需要 `promise_type` 成员, 其接口指定了自定义协程本身行为的方法。库编写者能够自定义调用协程时发生的事情, 当协程返回时发生的事情(通过正常方式或通过未处理的异常), 并自定义 `coroutine` 中任何表达式的行为。

2、`Promise` 对象需要提供哪些函数?

1)、`promise.initial_suspend()` 函数: 在协程框架被初始化并获得返回对象后挂起。

2)、`promise.get_return_object()` 函数: 在协程挂起时返回给调用者结果, 并把结果保存为一个局部变量。

3)、`promise.unhandled_exception()` 函数：在协程内捕获异常并在捕获区域内调用。

4)、`promise.final_suspend()` 函数：在协程结束后暂停，允许协程执行一些逻辑，并在协程执行完成和协程帧被销毁前挂起。

5)、`promise.return_void()` 函数：按创建的相反顺序销毁所有具有自动存储时间的变量。

6)、`promise.yield_value()` 函数：暂停协程的进行，并将变量存储下来。

3、Awaitable object 需要提供哪些接口？

1)、`await_ready()` 函数：在知道操作将同步完成而不需要暂停的情况下避免操作的成本。

2)、`await_suspend()` 函数：使协程被挂起，一旦协程被挂起就可以被恢复或销毁。

3)、`await_resume()` 函数：使被挂起的协程恢复运行，同时也可以抛出异常当到达协程外。

4、Coroutine handle 通常需要提供哪些函数？

1)、`coroutine_handle::resume()` 函数：恢复协程时，实现挂起的协程主体的其余部分。然后，被调用的主体函数可以查找挂起点索引并跳转到控制流中的适当点。

2)、`coroutine_handle::done()` 函数：查询当前挂起点是否为最终挂起点。

3)、`coroutine_handle::destroy()` 函数：调用适当的逻辑来销毁当前挂起点所有范围内的对象。

4)、`coroutine_handle::address()` 函数：转移 `coroutine_handle` 到相应的地址。

5)、`coroutine_handle::from_address()` 函数：从相应的地址转移 `coroutine_handle`。

5、为什么说 `co_yield` 和 `co_return` 是 `co_await` 的语法糖？

因为 `co_yield<expr>` 等价于 `co_await promise.yield_value(expr)`；`co_await` 等价于 `co_await promise.return_void()` 或者 `promise.return_value(<expr>)`，并且使协程执行 `promise.final_suspend()`。

6、简述协程函数的调用过程并阐述上述每个接口函数的功能。

协程函数首先构造 `promise_type` 对象，并通过 `promise.get_return_object()` 函数获取关联的返回对象，之后协程在 `promise.initial_suspend()` 函数处挂起，在协程内可以通过 `co_await` 挂起协程，也能通过 `co_yield` 挂起协程并返回一个值，如果在协程中遇到异常会调用 `promise.unhandled_exception()` 函数将其捕获，协程执行完毕或者遇到 `co_return` 语句时，会在 `promise.final_suspend()` 函数处挂起，最后调用 `coroutine_handle::destroy()` 函数销毁协程。

generator.h:

`promise_type` 部分：

```
struct promise_type
{
    Value value_;
    std::suspend_always initial_suspend() { return {}; }
    std::suspend_always final_suspend() noexcept { return {}; }
    std::suspend_always yield_value(Value val) {
        value_ = val;
        return {};
    }
    void return_void() { return; }
    generator get_return_object()
    {
        return generator{std::coroutine_handle<promise_type>::from_promise(*this)};
    }
    void unhandled_exception() { std::terminate(); }
};
```



首先需要有一个 `value` 成员来记录 `yield_value` 函数执行时参数的值, `initial_suspend` 和 `final_suspend` 函数分别在协程框架初始化后和协程结束后挂起, 因而直接 `return {}` 即可, 而 `yield_value` 则需要将成员赋值为参数后再挂起, `return_void` 函数没有参数直接返回即可, `get_return_object` 函数通过调用 `from_promise` 函数并以当前对象为参数来返回一个协程, `unhandled_exception` 函数通过调用 `terminate` 函数来捕获并处理异常。

generator 析构函数部分:

```
~generator() noexcept {
    if (coro_)
    {
        coro_.destroy();
    }
}
```

先判断协程是否存在, 若存在直接调用 `destroy` 函数销毁协程即可。

iterator 部分:

```
struct sentinel {};
```

首先在前面增设一个 `sentinel` 哨兵结构代表结尾。

```
friend bool operator==(const iterator& o, sentinel) noexcept
{
    if(o.coro_ == 0)
    {
        return true;
    }
    if(o.coro_.done())
    {
        return true;
    }
    return false;
}
```

`==` 操作符判断当前迭代器是否终止, 如果迭代器不存在或者运行结束即 `done` 函数返回值为真则终止了返回 `true`, 否则返回 `false`, 由于 `sentinel` 代表结束是 `done` 函数的判断条件因而也要作为参数。

```
friend bool operator!=(const iterator& o, sentinel) noexcept
{
    if( o == sentinel{})
    {
        return false;
    }
    return true;
}
```

`!=` 操作符则刚好相反, 如果迭代器与结束的 `sentinel` 相等则返回 `false` 否则返回 `true`。

```
iterator& operator++()
{
    coro_.resume();
    return *this;
}

iterator operator++(int)
{
    (void) operator++();
}
```

`++`和`++(int)`操作符中`++`是自增操作使迭代器指向下一个值，即让上一次协程挂起之后恢复运行，采用 `resume` 函数来实现，之后返回`*this` 来得到迭代器，而`++(int)`则是把`++`和变量的相对顺序调整了一下，这里功能是一样的，直接调用`++`操作符即可。

```
reference operator*() const noexcept
{
    return coro_.promise().value_;
}

pointer operator->() const noexcept
{
    return std::addressof(operator*());
}
```

`*`操作符直接返回 `promise` 的 `value_`成员即可得到迭代器的值，而`->`操作符用 `addressof` 函数取得相应的地址即可。

```
private:
    friend generator;

    explicit iterator(std::coroutine_handle<promise_type> coro) noexcept : coro_(coro) {}
    std::coroutine_handle<promise_type> coro_ = nullptr;
};
```

迭代器构造函数直接以相应的协程为参数构造并把其初始化为空即可。

```
iterator begin() noexcept
{
    if (coro_)
    {
        coro_.resume();
    }
    return iterator{coro_};
}

sentinel end() noexcept
{
    return {};
}
```

`begin()`迭代器在协程存在的情况下恢复协程运行并返回相应协程的迭代器，而`end()`迭代器返回的是尾迭代器后的一个迭代器，是 `sentinel` 类型的，并且没有具体内容。

```
private:
    explicit generator(std::coroutine_handle<promise_type> h) noexcept : coro_(h) {}
    std::coroutine_handle<promise_type> coro_ = nullptr;
```

`generator` 构造函数和迭代器一样以相应协程为参并把其初始化为空。

完成上述代码后，进入 `libco/libco_v4` 文件夹下用 `make` 命令编译代码，使用 `./main` 命令运行：

```
yuhaoshen@LAPTOP-QU516I60:/mnt/c/libco-handout/libco - TODO/libco_v4$ make
g++-11 -std=c++20 -o main main.cpp
yuhaoshen@LAPTOP-QU516I60:/mnt/c/libco-handout/libco - TODO/libco_v4$ ./main
Hello, ICS 2023!
2 3 5 8 13 21 34 55 89 144
```

运行结果正常，成功通过测试，完成 part4。

## 七、part5:

`generator.h`:

`promise_type` 部分:

```

struct promise_type
{
    promise_type* node_;
    promise_type* prev_ = nullptr;
    Value value_;
    std::suspend_always initial_suspend() { return {}; }
    std::suspend_always final_suspend() noexcept { return {}; }
    std::suspend_always yield_value(Ref&& x) noexcept {
        node_ -> value_ = std::move(x);
        return {};
    }
    std::suspend_always yield_value(Ref& x) noexcept {
        node_ -> value_ = x;
        return {};
    }
    void return_void() { return; }
    generator get_return_object()
    {
        return generator{std::coroutine_handle<promise_type>::from_promise(*this)};
    }
    void unhandled_exception() { std::terminate(); }
    promise_type() : node_(this) {}
};

```

由于新实现的 generator 要能够可以递归，因而要维护一棵调用树，在成员中新加入两个变量 node\_ 和 prev\_ 分别指向当前结点的协程和他的双亲结点协程，其成员函数与 part4 中部分大致相同，但对 yield\_value 函数需要做出修改，首相用 move 函数将 x 绑定到 node\_ -> value\_ 上之后再对其进行赋值，同时在其中加入构造函数并将当前结点协程初始化为自己。为了实现调用树的功能，还需要在成员中加入 final\_awaiter 和 yield\_awaiter 结构分别实现向上调用和向下调用的功能。

```

struct final_awaiter
{
    bool await_ready() noexcept { return false; }

    std::coroutine_handle<> await_suspend(std::coroutine_handle<promise_type> h) noexcept
    {
        auto curr = h.promise().node_;
        auto prev = h.promise().prev_;
        if (prev)
        {
            curr -> node_ = prev;
            return std::coroutine_handle<promise_type>::from_promise(*prev);
        }
        return std::noop_coroutine();
    }

    void await_resume() noexcept {}
};

```

final\_awaiter 结构实现向上调用功能首先需要有一个 await\_ready 函数告诉 co\_await 是否准备好了，其直接返回 false 将协程挂起，并在之后根据 await\_suspend(h) 句柄的返回值类型进行下一步操作，在句柄中开两个变量记录当前结点的协程和其双亲结点的协程，如果其双亲结点的协程存在则将当前结点协程赋值为其双亲结点协程并且返回其双亲结点协程的句柄来恢复其双亲结点，从而实现向上调用的效果，而如果不存在的话则返回 noop\_coroutine 函数表明已经回到调用树的根结点。同时在结构体中还需要一个 await\_resume 函数来实现对于协程的恢复。

```

struct yield_awaiter
{
    generator gen_;

    explicit yield_awaiter(generator &&g) noexcept: gen_(std::move(g)) {}

    bool await_ready() noexcept
    {
        return !gen_.coro_;
    }

    std::coroutine_handle<> await_suspend(std::coroutine_handle<promise_type> h) noexcept
    {
        auto &origin = h.promise();
        auto &nested = gen_.coro_.promise();
        auto &curr = origin.node_;
        nested.node_ = curr;
        curr->node_ = &nested;
        nested.prev_ = &origin;

        return gen_.coro_;
    }

    void await_resume() noexcept {}
};

```

yield\_awaiter 结构实现向下调用功能首先需要一个生成器成员来获取当前结点的协程和其子女结点的协程，其构造函数采用 move 函数将其与生成器参数进行绑定来进行复制构造，其 await\_ready 函数与 final\_awaiter 结构有些许不同，只有在生成器子女结点协程存在时才返回 false 进行暂停，在句柄 await\_suspend(h) 中执行的操作与 final\_awaiter 结构正好相反，将其子女结点当前协程的值通过取地址的方式与其自身关联，并将其子女结点的双亲结点协程赋值为自身，最后返回子女结点协程来对其进行恢复，从而实现向下调用的效果。同 final\_awaiter 结构一样需要一个 await\_resume 函数来对协程进行恢复。

```

yield_awaiter yield_value(generator&& g) noexcept
{
    return yield_awaiter{std::move(g)};
}

void resume()
{
    std::coroutine_handle<promise_type>::from_promise(*node_).resume();
}

```

返回生成器的 yield\_value 函数以及恢复协程的 resume 函数分别直接调用 yield\_awaiter 的构造函数以及当前结点协程的 resume 函数即可。

```

generator(generator&& other) noexcept: coro_(std::exchange(other.coro_, {})) {}

```

新增的含参构造函数通过调用 exchange 函数进行移动构造防止创建一个新资源。iterator 部分：

```

iterator& operator++()
{
    coro_.promise().resume();
    return *this;
}

```

大体上与 part4 部分相同，只有 ++ 操作符需要将要恢复的协程变为协程的 promise 对象需要修改。



sleep.h:

sleep 部分:

```
struct sleep {
    sleep(int n_ms) : delay{n_ms} {}

    std::chrono::milliseconds delay;
    bool await_ready() noexcept { return false; }
    void await_suspend(std::coroutine_handle< h> h) noexcept
    {
        task_queue.pop();
        std::this_thread::sleep_for(delay);
        task_queue.push([h]() -> bool
        {
            h.resume();
            return h.done();
        });
    }
    void await_resume() noexcept {}
};
```

sleep 类的功能是实现一个延时执行的功能，在模板的基础上同 final\_awaiter 和 yield\_awaiter 一样需要 await\_ready, await\_suspend 和 await\_resume 三个函数，其中只有 await\_suspend 函数需要修改，为了实现延时的功能，首先应将其挂起，由于 task\_queue 中按顺序存放要依次执行的协程，当前执行的协程应该在队首，因而将队首元素弹出能做到挂起的效果，之后调用 sleep\_for 函数来延时 delay 时间，最后将参数协程压入队尾并恢复执行，将其函数参数赋值为当前协程是否执行完毕的结果，以便判断所有协程是否均执行完毕。

Task 部分:

```
struct promise_type
{
    std::suspend_always initial_suspend() { return {}; }
    std::suspend_always final_suspend() noexcept { return {}; }
    std::suspend_always yield_value(std::function<bool()> func) {
        task_queue.push(func);
        return {};
    }
    void return_void() { return ; }
    Task get_return_object()
    {
        return Task{std::coroutine_handle<promise_type>::from_promise(*this)};
    }
    void unhandled_exception() { std::terminate(); }
};
```

promise\_type 和之前在 generator 中的大体一样，只有在 yield\_value 函数中要通过将给定的参数压入队尾来实现赋值的效果。

```
std::coroutine_handle<promise_type> coro;

Task(std::coroutine_handle<promise_type> h) : coro(h) {
    task_queue.push([this]() -> bool
    {
        coro.resume();
        return coro.done();
    });
}

~Task()
{
    if (coro)
    {
        coro.destroy();
    }
}
```

实现构造函数则首先需要加入一个协程成员，之后同 `await_suspend` 函数一样将该成员初始化为参数然后将参数协程压入队尾并将其函数参数赋值为当前协程是否执行完毕的结果，而析构函数则同样是在协程成员存在的时候调用 `destroy` 函数销毁协程。

`wait_task_queue_empty` 函数部分：

```
void wait_task_queue_empty() {
    while (!task_queue.empty())
    {
        std::function<bool()> func = task_queue.front();
        if (func())
        {
            task_queue.pop();
        }
    }
}
```

直接每次判断队首协程是否执行完成，如果执行完成就出队，由于执行该函数时协程均已执行完成，因而能将队列置空。

完成上述代码后，进入 `libco/libco_v5` 文件夹下用 `make` 命令编译代码，使用 `./main` 命令运行：

```
yuhaoshen@LAPTOP-QU516I60:/mnt/c/libco-handout/libco - TODO0/libco_v5$ make
g++-11 -std=c++20 -o main main.cpp
yuhaoshen@LAPTOP-QU516I60:/mnt/c/libco-handout/libco - TODO0/libco_v5$ ./main
Start libco_v5 test
libco_v5 task test passed!
```

成功通过测试，完成 part5 至此完成本次实验所有内容。